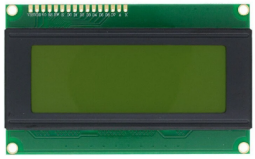
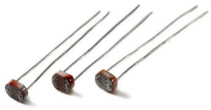
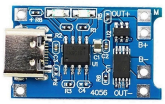
Group 3
Altobar, Jim Clet N.
Dungo, Loriedelle V.
Jordan, Jien R.

Week 1: Current Progress Assessment and Mathematical Enhancement

Project Status Report (50% Completion)

Hardware Inventory

ESP 32	
LCD 20*4 (I2C interface)	
Solar Panel 6v	
LDR	
Buzzer	

TP4056	
2x Battery Lithium-Ion 18650 3.7v	
Battery Holder (Parallel)	

Implemented Software Features

- LCD initialization and updates

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 20, 4);  // LCD at I2C address 0x27, 20x4 characters

lcd.begin();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print(" Solar Monitor  ");
```

  - LiquidCrystal_I2C library controls the LCD via I2C
  - lcd.begin() starts communication
  - lcd.backlight() turns on the LCD backlight
  - lcd.setCursor(col, row) sets the text position
  - lcd.print() to display data like irradiance, voltage and LDR readings
- Smoothed ADC readings

```
float readSmoothedADC(int pin) {
  int total = 0;
  for (int i = 0; i < AVG_WINDOW; i++) {
    total += analogRead(pin);
    delay(5);
  }
  return total / float(AVG_WINDOW);
}
```

  - analogRead(pin) reads raw analog voltage (0–4095 on ESP32)
  - function reads multiple times (AVG_WINDOW = 10) and averages the result to reduce noise
  - used for more stable sensor input
- Voltage, power and irradiance calculation

```
float calculateVoltage(float adcVal) {
  return (3.3 * adcVal) / 4095.0;
}

float calculatePower(float voltage) {
  return (voltage * voltage) / R;
}

float calculateIrradiance(float power) {
  float solarPower = (power * 100.0) / EFFICIENCY;
  return solarPower / EMVADO;
}
```

- Voltage: Converts ADC value to actual voltage
- Power: Assumes voltage across a known resistor to compute power: $P = V^2 / R$
- Irradiance: Converts electrical power to solar irradiance (W/m^2), considering panel area and efficiency

- Irradiance derivative (rate of change)

```
float calculateDerivative(float current, float previous, float dt) {
  return (dt > 0) ? (current - previous) / dt : 0;
}
```

- Calculates how fast the irradiance is changing (dI/dt) using difference in values over time (dt in seconds) and useful for tracking rapid changes

- Buzzer feedback logic

```
void buzzerFeedback(float irradiance) {
  if (irradiance < 200) {
    digitalWrite(BUZZER_PIN, LOW);
  } else if (irradiance < 600) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(100);
    digitalWrite(BUZZER_PIN, LOW);
  } else {
    for (int i = 0; i < 2; i++) {
      digitalWrite(BUZZER_PIN, HIGH);
      delay(100);
      digitalWrite(BUZZER_PIN, LOW);
      delay(100);
    }
  }
}
```

- Controls buzzer output based on irradiance:
  - < 200 → No buzz.
  - 200–599 → Single short beep.
  - 600+ → Two quick beeps.
- gives real-time audible feedback on sunlight strength

- Serial output for debugging

```
  Serial.println("=== Measurement ===");
  Serial.print("Voltage (V): "); Serial.println(voltage, 3);
  Serial.print("Power (mW): "); Serial.println(power * 1000, 2);
  Serial.print("Irradiance (W/m^2): "); Serial.println(irradiance, 1);
  Serial.print("dI/dt: "); Serial.println(derivative, 2);
  Serial.print("LDR Value: "); Serial.println(ldrValue);
```

- Displays data on the Serial Monitor for testing and monitoring
- Includes
  - voltage
  - power
  - irradiance
  - derivative
  - LDR value

- Main Loop

```
float solarADC = readSmoothedADC(SOLAR_ADC_PIN);
float voltage = calculateVoltage(solarADC);
float power = calculatePower(voltage);
float irradiance = calculateIrradiance(power);
float derivative = calculateDerivative(irradiance, lastIrradiance, dt);
```

  - Runs continuously to update readings every 2 seconds (delay(REFRESH_INTERVAL))
  - Reads smoothed ADC → converts to voltage → calculates power and irradiance
  - Measures change rate (derivative) and updates the LCD + buzzer

Challenges & Roadblocks

- No real-time wireless data transmission
- Hardcoded ADC range with ESP32
- No calibration or validation of sensor readings

Updated Timeline

Week	Goal
Week 1	Implemented core software features including LCD display updates, smoothed ADC readings, irradiance calculation and buzzer feedback logic. Basic hardware components such as the ESP32, LDR, solar panel, and buzzer were integrated and tested, achieving approximately 50% project completion
Week 2	Build the web application with backend API, simple webpage layout, and basic graphs for sensor data and also start using Fourier and Laplace transform methods for signal analysis and system modeling.
Week 3	Improve signal processing with advanced filtering using Fourier and Laplace methods, complete the web application with real-time data and controls and begin full system integration with testing and performance checks.
Week 4	Finalize the web application with full features and perform detailed mathematical analysis using Fourier and Laplace methods, and conduct thorough system testing to validate performance, reliability and power efficiency.
Week 5	Demonstrate the full system with all features, complete comprehensive documentation of hardware, software, and mathematical techniques, and present the final project with performance analysis and a live system showcase.

Mathematical Foundation Enhancement

Fourier Series Analysis

- Data to analyze: irradiance and ldrValue
- Use case: Identify periodic patterns (e.g., sunlight variation)
- Implementation Plan:
  - Sample irradiance at uniform intervals
  - Use FFT to extract dominant frequency components
  - Design low-pass filter if noise dominates high-frequency bands

Laplace Transform & Control Systems

- System element: Irradiance tracking and buzzer control
- Transfer function modeling:
  - Input: Sunlight intensity
  - Output: Irradiance measurement (processed)
  - Transfer function (idealized):
    G(s)=Kτs+1G(s) = \frac{K}{\tau s + 1}
    where τ = sensor/system response time
- Stability:
  - Currently open-loop
  - Add PI/PID controller if aiming for active feedback system (e.g., solar tracking)

Hardware Assessment & Enhancement Plan

Evaluation

- Solar panel + voltage sensing for irradiance
- LDR for light comparison
- Buzzer for feedback

Required Additions

- Possibly ESP32 for better performance
- Replace the charging module to TP5100

Revised Wiring Plan

- Add pull-down resistors if floating pins
- Shield analog lines to reduce noise

Power Consumption

- LCD 5v 20*4
  - Create an alternate source line solely for LCD
- analog reads, and buzzer are low-power
- Optimize by:
  - Using deep sleep in future
  - Minimizing LCD refresh rate

Data Collection Strategy

Plan

- Store
  - irradiance
  - ldrValue
  - voltage
  - timestamp
- Collect data through web app

Dataset Organization

- CSV format:
  - timestamp
  - irradiance
  - voltage
  - ldrValue
  - change

Preprocessing

- Normalize values
- Remove spikes (use moving average or median filter)
- Interpolate missing data

Validation

- Visualize data for anomalies
- Set thresholds for outlier removal

updated GitHub repository:

https://github.com/schif3r/Irridiance_Meter.git